OCR - PROJET 10

Mettez en place un pipeline d'orchestration des flux

Mathilde LE SOLLIEC
04-2026

SOMMAIRE

1 Contexte

2 Logigramme

3 Choix des outils et installations

4 Résultats

1. Contexte

CONTEXTE

Entreprise
BottleNeck, **un marchant de vin**.

Projet
Nettoyage des données (excel) provenant des 2 systèmes CMS et ERP.
Réconciliation de ces données afin :

- de calculer le chiffre par produit et le chiffre d'affaires total réalisé.
- Identification des vins premium avec utilisation des méthodes statistiques tel que le z-score et l'intervalle interquartile.

OBJECTIF DE LA MISSION

- En tant Data Engineer : **automatiser cette chaîne de traitement** et d'analyse de données.
- Les responsables de données doivent recevoir les rapports des chiffres d'affaires tous les mois ainsi que les **extractions des fichiers** contenant les vins premium et les vins ordinaires.

SOURCES DE DONNEES

Liaison

Fait le lien entre les product_id et les id_web

Erp

Une ligne par type de produit (product_id)
avec le prix, la quantité de stock et le statut du stock (outoftock ou instock) et s'il est vendu sur le web (0 ou 1)

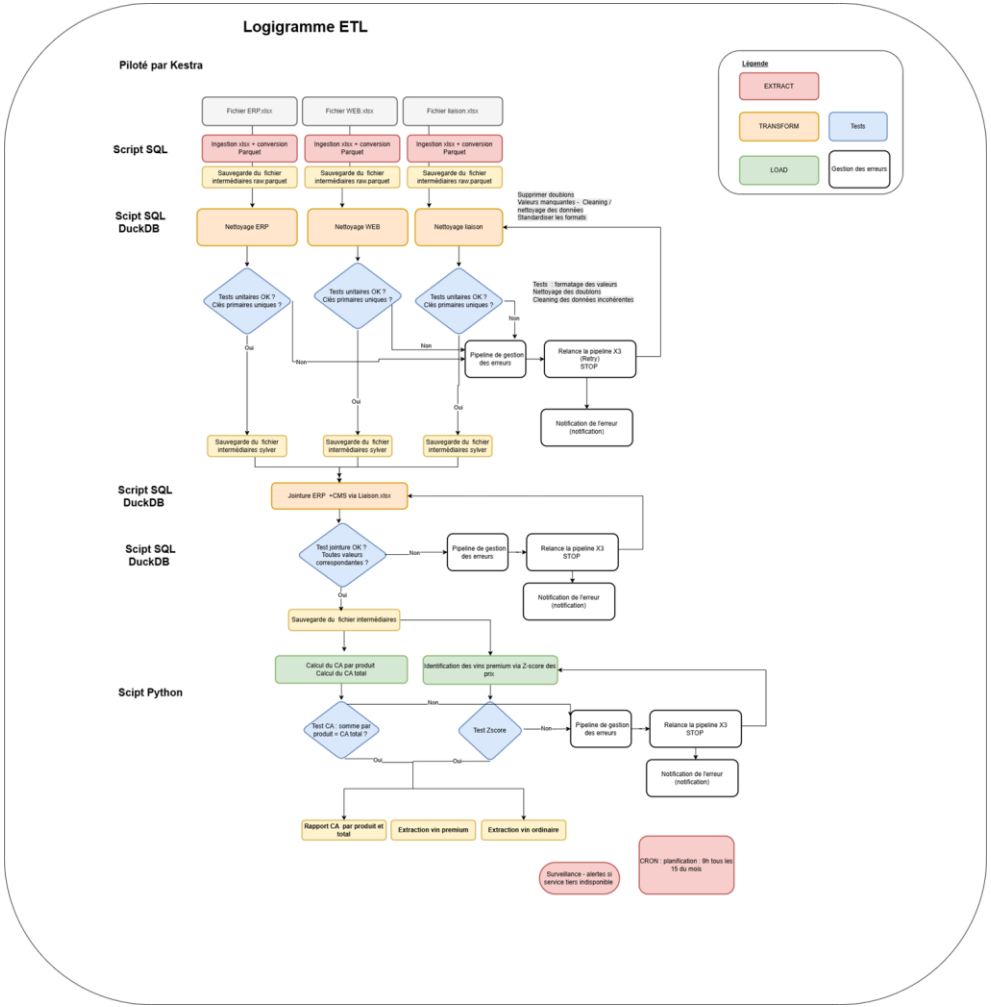
Web

Le ligne par produit (id_web)
Fiche complète des produits pour WooCommerce, avec tout ce qui concerne la gestion, la visibilité, le contenu, les ventes et la fiscalité.

2. Logigramme

Création d’un logigramme

Visualiser et structurer l’ensemble du pipeline ETL avant de coder, afin de clarifier les étapes et éviter les erreurs de logique.

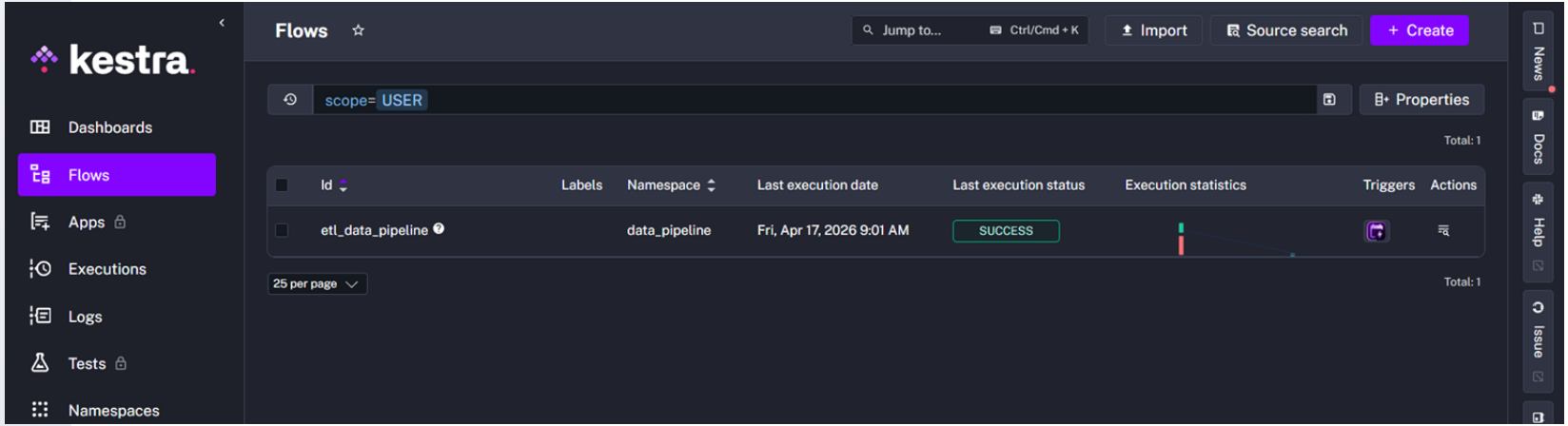


3. Choix des outils et installations

Kestra

Kestra est un outil d'orchestration de pipelines de données qui permet de définir, automatiser et monitorer des workflows ETL de manière déclarative via du YAML.

- Installé via Docker avec Docker compose en utilisant l'image officielle (image: kestra/kestra:v1.0.34)
- Orchestration avec Kestra : création d'un Flow YAML (id, namespace, tasks).
- Interface exposé sur http://localhost:8080



Kestra

- **Data Stack**
  - DuckDB : DuckDB permet de faire du SQL "In-Process" sans base de données à installer. Lire les fichiers (parquet csv, excel) et transformations simples et rapides.
  - Python : logique custom (z-score)

- **Principales Tasks utilisées :**
  - **Tasks de code :**
    - io.kestra.plugin.jdbc.duckdb.Query
    - io.kestra.plugin.scripts.python.Script
  - **Tasks d'orchestration**
    - io.kestra.plugin.core.flow.Parallel → Gain de temps en exécutant plusieurs tâches simultanément.)
    - io.kestra.plugin.core.flow.Sequential → Sécurisation de l'enchaînement logique des étapes.)
    - io.kestra.plugin.core.flow.If → Branchement conditionnel selon les résultats (Décision).)
  - **Tasks de Core**
    - io.kestra.plugin.core.log.Log → Traçabilité et monitoring en temps réel des étapes du workflow.)

4. Résultats

Objectifs

1. La pipeline se lance automatiquement
2. Les tâches sont orchestrés en simultanées
3. Mettre en place des tests automatisés (unitaires et d’intégration)
4. Bonne gestion des erreurs
5. Les résultats sont cohérents avec ce qui est attendu

Objectif

1. La pipeline se lance automatiquement

Planification de l'exécution de l'ensemble du workflow tous les 15 du mois à 9h en utilisant le mécanisme du trigger et du cron :

Extrait du flow

```
triggers:
  - id: monthly_schedule
    type: io.kestra.plugin.core.trigger.Schedule
    cron: "0 9 15 * *"
```

Objectif

2. Orchestraction de tâches en simultanées

Utilisation des tâche conteneurs qui exécutes ses sous tâche en en parallèle ou séquentielle

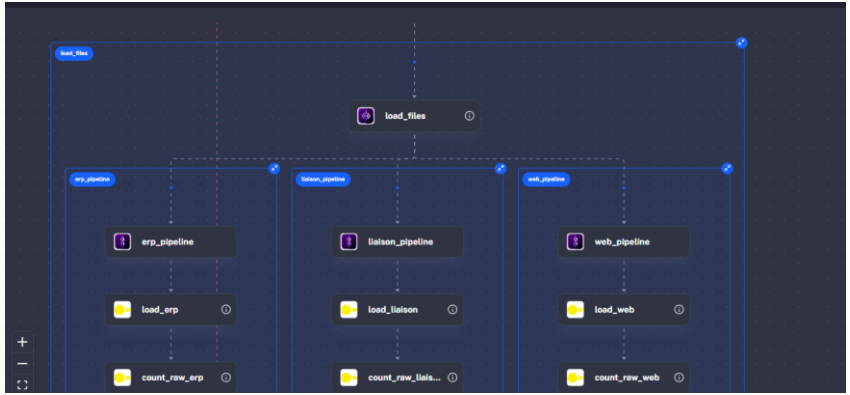
Extrait du flow

```yaml
- id: load_files
  type: io.kestra.plugin.core.flow.Parallel
  description: "Chargement des fichiers Excel en Parquet"
  tasks:

    # — Fichier LIAISON ————————————————————————————————————
    - id: liaison_pipeline
      type: io.kestra.plugin.core.flow.Sequential
      tasks:
```

Utilisé pour le téléchargement et cleaning des fichiers avant de les fusionner

Extrait de la topology



Objectif

3. Mettre en place des tests automatisés (unitaires et d'intégration)

1. Fiabilité des Sources (Nettoyage)

- Dédoublonnage : Suppression des doublons sur les clés (product_id, sku, id_web).
- Vérification qu'il n'y a pas de clé primaire Null

2. Intégrité de la Fusion (Jointures)

- Absence de NULL après jointure
- Nombre de ligne ne gonfle pas après la jointure

3. Cohérence des calculs (CA et Zscore)

- Cohérence Financière : Validation du Chiffre d'Affaires (ne peut être ni nul, ni négatif).
- Validation Statistique (Python) :
  - Test de la distribution du Z-Score (moyenne ≈ 0, écart-type ≈ 1).
  - Vérification de l'existence de "Millésimes"

Objectif

Bonne gestion des erreurs

- **Echec de la pipeline**

Gestion d'erreurs locale d'une tâche : ici si un des tests parametrés echoux, la pipeline s'arrête

Extrait du flow

```
- id: assert_liaison
  type: io.kestra.plugin.core.flow.If
  description: "Bloque l'export si des doublons détectés"
  condition: "{{ outputs.test_liaison.row.doublons_lignes > 0 }}"
  then:
    - id: fail_liaison
      type: io.kestra.plugin.core.execution.Fail
      errorMessage: |
        ❌ Tests liaison échoués :
          - Lignes doublons : {{ outputs.test_liaison.row.doublons_lignes }}
```

Parametrage d'un log pour savoir où se trouve l'erreur :
Global : Gestion d'erreurs globale d'un flux qui doit se trouver à la racine du flux

Extrait du flow

```
errors:
  - id: pipeline_failed
    type: io.kestra.plugin.core.log.Log
    message: |
      🚨 Pipeline échoué :
        - Flow       : {{ flow.namespace }}.{{ flow.id }}
        - Exécution  : {{ execution.id }}
        - Tâche      : {{ errorLogs()[0]['taskId'] }}
        - Message    : {{ errorLogs()[0]['message'] }}
```

Objectifs

Bonne gestion des erreurs

- **Retry x3**

Si échec de la tâche : relance 3 fois la tâche à 5 secondes d'intervales

Extrait du flow

```yaml
pluginDefaults:
  - type: io.kestra.plugin.jdbc.duckdb.Query
    values:
      retry:
        type: constant
        maxAttempt: 3
        interval: PT5S
```

Objectifs
Résultat final cohérent avec ce qui est attendu

- Portez attention aux résultats des analyses effectuées par notre Data Analyst, Stéphane :

  - Fichier fusionné : 714 lignes
  - Le chiffre d'affaires total réalisé est de 70 568.60 euros
  - Nombre de vins millésimes détectés : 30

Extrait des logs

```
2026-04-10 13:04:29.764  ✅ Pipeline ETL terminé avec succès
📊 Liaison : 825 lignes exportées
📊 ERP     : 825 lignes exportées
📊 WEB     : 714 lignes exportées
📊 FUSION  : 714 lignes exportées
RÉSULTAT FINANCIER :
💰 Chiffre d'Affaires Total : 70568.6 €
🍷 CLASSIFICATION DES VINS :
Vins millésimes  : 30 vins
Vins ordinaires  : 684 vins
```

Fin du projet

Nouvelles compétences et réalisations :

- Lier github avec kestra pour enregistrer automatiquement (création d'un token git)
- Scheduler CRON
- Paramétrage d'un message qui dit où la pipeline s'est arreté

Difficultés :

- Découpage sous forme de tache : à quel point découper les tâches
- Choisir le type de tache dans kestra (documentation).
- Adapter les sources entre les scripts en local et les dockers (mettre en volume les sources pour qu'il puisse les retrouvers).
- Docker image : ne pas mettre la version latest mais une version fixe sinon bug sur les mise à jours !

Suite du projet :

- Paramétrer un Message aux équipes pour envoyer un mail en cas d'erreur de la pipeline
- Utiliser des outils de tests (ex pytest)